

Monitoramento de temperatura usando BLE e ESP32



1. Visão Geral da atividade

O objetivo desta atividade é desenvolver um sistema embarcado utilizando o microcontrolador **ESP32** que atua como um servidor **BLE (Bluetooth Low Energy)**. O sistema irá monitorar variáveis ambientais (temperatura e umidade), exibir informações em um display LCD local, receber comandos de botões físicos e controlar atuadores (LEDs e LED RGB). Deve-se montar uma protoboard com os elementos de hardware descritos no item 2).

Um aplicativo de smartphone (Cliente BLE) será utilizado para monitorar os dados em tempo real (incluindo gráficos históricos), controlar os LEDs e configurar o dispositivo de forma segura através de pareamento por senha.

2. Arquitetura de Hardware

- **Processador:** ESP32
- **Sensores:** 1x Sensor de Temperatura e Umidade DHT22 (com resistor de pull-up)
- **Interface de Saída Visual:**
 - 1x Display LCD
 - 1x LED RGB (Anodo ou Catodo Comum)
 - 2x LEDs simples (Ex: LED Vermelho e LED Verde)
- **Interface de Entrada Física:**
 - 2x Botões do tipo *Push-Button* (com resistores *pull-down*).
 - 4x Botões do tipo *Switch-Button* (Chaves estáticas Liga/Desliga, com resistores *pull-down*).

3. Especificação das Funcionalidades Locais (Firmware ESP32)

3.1. Funcionalidades do Display LCD

O display deve alternar suas telas automaticamente (ex: a cada 3 segundos) ou através da interação com os botões. As informações exibidas devem ser:

- **Tela 1 (Atual):** Temperatura atual em Celsius (°C) e Umidade Atual (%).
- **Tela 2 (Fahrenheit):** Temperatura atual in Fahrenheit (°F) e Umidade Atual (%).
- **Tela 3 (Histórico):** Valores Mínimos e Máximos registrados de Temperatura desde a inicialização.
- **Tela 4 (Histórico):** Valores Mínimos e Máximos registrados de Umidade desde a inicialização.
- **Tela 5 (Status):** Estado da conexão BLE (Desconectado / Anunciando / Conectado) e valor RSSI da conexão (usar: `int rssi = NimBLEDevice::getPeerRSSI(atualConnHandle);`).

3.2. Funcionalidades do Botões Físicos

- **Push-Button 1:** Faz a alternância manual da tela do LCD;
- **Push-Button 2:** Reseta as memórias de valores Mínimos e Máximos (zera o histórico);
- **4x Switch-Buttons:** O estado de cada chave deve ser monitorado. Suas funções serão:
 - *Switch 1:* Bloqueia/Libera o controle dos LEDs pelo smartphone (Controle Local vs. Remoto);
 - *Switch 2, 3:* Ligam/Desligam os led localmente (mudança deve ser refletida no APP);
 - *Switch 4:* Define a visualização do gráfico no APP entre °C e F.

3.3. Funcionalidades dos Atuadores (LEDs)

- **LEDs Simples:** Devem responder aos comandos enviados pelo aplicativo (Ligar/Desligar) ou pelos switches 2 e 3.
- **LED RGB:** Deve misturar as cores (via PWM no ESP32) correspondentes aos valores de Vermelho (R), Verde (G) e Azul (B) enviados pelo APP.

4. Especificação da Camada Bluetooth (BLE)

Para atender aos requisitos de performance e segurança personalizados, a pilha BLE será configurada fora dos padrões automáticos do ecossistema.

4.1. Parâmetros de Conexão Customizados

Para otimizar o consumo de energia e o tempo de resposta, os parâmetros de rádio serão fixados em valores específicos fora do *default*:

- **Advertising Interval (Intervalo de Anúncio):** Configurado para **400 ms** (reduz o consumo do ESP32 enquanto espera conexões, sacrificando levemente a velocidade com que o celular detecta o dispositivo ao abrir o app).
- **Connection Parameters:**
 - *Minimum Connection Interval:* **50 ms**
 - *Maximum Connection Interval:* **100 ms** (Garante uma taxa de atualização ágil para os gráficos do app).
 - *Slave Latency:* **0** (O ESP32 responderá a todo evento de conexão do celular, evitando atrasos nos comandos dos LEDs).
 - *Supervision Timeout:* **2000 ms** (2 segundos para detectar queda abrupta de conexão).

4.2. Segurança e Autenticação

- **Método de Pareamento:** **Passkey Entry**.
- O ESP32 exigirá criptografia com autenticação (MIMT - *Man-In-The-Middle Protection*).

- Ao tentar conectar, o usuário deverá digitar uma senha estática de 6 dígitos no smartphone (definida no código do ESP32, ex: 123456). Se a senha estiver incorreta, a conexão é rejeitada pelo módulo de segurança do ESP32.

5. Arquitetura do Perfil GATT (Serviços e Características)

Para organizar os dados que trafegam entre o ESP32 e o APP, utilize as seguintes UUIDs e propriedades:

Serviço / Característica	Propriedades	Funcionamento
Serviço 1: Monitoramento Ambiental (UUID: 0x181A - Environmental Sensing)	-	Agrupar os dados coletados pelo sensor DHT22.
└─ Característica 1: Dados Atuais	Read, Notify	Envia um array de bytes contendo a Temperatura em Celsius, Fahrenheit e a Umidade. O aplicativo ativa o <i>Notify</i> para receber atualizações a cada segundo.
└─ Característica 2: Gráfico Histórico	Read	O ESP32 mantém internamente um vetor de médias dos últimos 60 minutos. Quando o app solicita a leitura, o ESP32 descarrega os dados históricos para o gráfico.
Serviço 2: Controle de Atuadores (UUID: Customizado de 128-bits)	-	Agrupar os comandos de controle dos LEDs periféricos.
└─ Característica 1: LEDs Simples	Read, Write	Permite ler o estado atual dos LEDs e enviar comandos de 1 byte (ex: bit 0 controla LED 1, bit 1 controla LED 2). Exige resposta de confirmação (GATT Response).
└─ Característica 2: LED RGB	Write Without Response	Envia uma sequência de 3 bytes representando as cores [R, G, B]. Utiliza <i>Write Without Response</i> para que a transição de cores através de um <i>Color Picker</i> seja fluida.

Serviço / Característica	Propriedades	Funcionamento
Serviço 3: Indicadores de conexão (UUID: Customizado de 128-bits)		
└─ Característica 1: RSSI (-dBm)	Read, Notify	Indicador de Intensidade do Sinal Recebido no ESP32
└─ Característica 1: contador de notificações	Read,	Obtém a quantidade de notificações enviadas pelo ESP32 no último minuto de conexão

Serviços e características adicionais podem ser incluídos caso seja necessário para atender as demais especificações.

6. Requisitos do Aplicativo Mobile (Cliente)

O aplicativo pode ser implementado em qualquer plataforma (MIT inventor, Thunkable, etc) ou linguagem (Dart/Flutter, React, C#) e deve rodar obrigatoriamente em Android 10+.

O aplicativo deve conter as seguintes seções funcionais:

- 1. Tela de Conexão:** Scanner BLE que filtra pelo nome do ESP32. Dispara o pop-up nativo do sistema operacional exigindo o código de pareamento (*Passkey*) apresentado no display LCD no momento da tentativa de conexão.
- 2. Painel de Monitoramento:**
 - Displays numéricos legíveis exibindo a temperatura atual (°C e °F) e umidade (%).
 - Dois componentes gráficos de linha (eixo X: Tempo / eixo Y: Valor) mostrando o comportamento da Temperatura e Umidade ao longo da última hora (armazenamento local no APP).
- 3. Painel de Controle:**
 - Dois botões do tipo *Toggle/Switch* virtuais para ligar/desligar os dois LEDs simples.
 - Um componente do tipo *Color Picker* (roda de cores) que converte a cor selecionada em valores de 0 a 255 (RGB) e envia imediatamente ao ESP32.
 - Um botão do tipo *Toggle/Switch* virtual para resetar os valores min/max para temperatura e umidade.
- 4. Painel de acompanhamento da conexão:**
 - Componente Gráfico de linha (eixo X: Tempo / eixo Y: RSSI) mostrando a intensidade do sinal recebido nos últimos 60 segundos.
 - Display numérico informando a quantidade total de notificações recebidas nos últimos 60 segundos.

8. Entrega

Data: 21/06/2026

Trabalho em dupla

O que entregar:

- Arquivo .apk (Android Package) para instalação direta;
- Link para o Wokwi com o mesmo esquema de ligações dos elementos da protoboard;
- Link do repositório no github com o código fonte (branch main + tag v1.0):
 - do firmware do ESP32
 - do aplicativo mobile
- Documentação do projeto no Github contendo:
 - Diagrama de classes e fluxograma do firmware (setup, loop e callbacks)
 - Tabela GATT com todos os UUIDs e propriedades (detalhamento da seção 5)
 - Diagrama de classes e descrição das principais funções/métodos.
- Instruções de compilação e instalação do APP